

AI Engineer Hiring

Job Description + First-Round AI Agent Assessment

1. Job Description

AI Engineer: AI Systems

CalQuity is building AI infrastructure for financial institutions, helping teams work with large volumes of financial data and make faster, better-informed decisions.

As an AI Engineer, you will build AI agents and multi-step workflows, develop retrieval and reasoning systems over large datasets, and design the APIs, data pipelines, and backend systems that power them.

We are a small team, so this role comes with a high degree of ownership. You will be expected to understand the problem, think through the right solution, build it, and make sure it works reliably in production.

What You'll Work On

- Build AI agents and multi-step workflows for financial-data use cases.
- Develop retrieval and reasoning systems over large datasets.
- Design and maintain the APIs, data pipelines, and backend systems that power them.
- Debug, test, deploy, and monitor reliable production systems.
- Work closely with product, customers, and other engineers to turn business problems into useful software.

What We're Looking For

- Strong engineering fundamentals.
- Curiosity, product thinking, and the ability to figure things out independently.
- Ability to understand ambiguous problems, choose sensible solutions, and take ownership end-to-end.
- Experience building AI agents, retrieval or reasoning systems, or multi-step workflows is a plus.
- Comfort working in a small, fast-moving team.

Experience: 0-2 years

Location: Remote

We care as much about ownership, curiosity, and sound product judgment as we do about technical skill.

Please note, we are looking for people to join us full time.

Compensation

As an early hire, you will receive a combination of cash compensation and equity.

Cash: ₹40,000-₹50,000 per month

Equity: 0.5%-1.0%

Total CTC: ₹8.8-14 lakh

2. First-Round Assessment

AI Agent Assessment: ParcelPilot Customer Support

Background

ParcelPilot is a B2B logistics platform used by businesses to book and manage shipments across multiple carrier partners. Its 20-person customer operations team handles hundreds of support requests every week.

Customers ask questions about account entitlements, customer-specific contract terms, shipment cancellations, service credits, and support SLAs. They also report product issues that may require investigation using order, account, or ticket data.

Today, the customer operations team manually searches across policies, product documentation, customer agreements, past support tickets, and structured operational data to resolve these requests.

ParcelPilot wants an AI support system with two user contexts: a customer-facing support agent that answers customer queries quickly and reliably, and internal support/operations workflows that help ParcelPilot investigate, prioritise, and act on issues across its support activity.

The source base is intentionally imperfect. Some documents are outdated, **customer-specific agreements may override general policies**, and historical ticket resolutions may contain

incorrect guidance. Your system should handle these situations deliberately rather than assuming that every source is equally reliable.

Candidate Data Pack

[Open the candidate data pack here.](#)

The pack contains:

- 01_Support_Policy_v3_CURRENT.pdf
- 02_Support_Policy_v2_DEPRECATED.pdf
- 03_Cancellation_and_Service_Credit_SOP_v4.pdf
- 04_Product_Operations_Guide_and_Known_Issues.pdf
- 05_Northstar_Logistics_Enterprise_Agreement.pdf
- 06_LumenWorks_Service_Agreement.pdf
- ParcelPilot_Assessment_Data.xlsx

The workbook contains ParcelPilot's internal account, order, and ticket data. This data may be used by both the customer-facing agent and authorised internal workflows, but access must be scoped appropriately so that customers cannot access information belonging to other accounts.

Use the dataset snapshot time stated in the workbook's **README** sheet as the reference time for all time-based questions.

Historical ticket resolutions should be treated as context only and may contain incorrect information.

Your Task

Build at least one AI chatbot for ParcelPilot. You may choose either a customer-facing support chatbot that answers customers' direct questions and escalates requests when necessary, or an internal support/operations chatbot that helps authorised ParcelPilot staff investigate customer issues, answer support questions, and work with operational data. You are welcome to support both user contexts.

You have flexibility in how you design the system. We care about both the technical implementation and the decisions you make about how the product should behave.

Feel free to make assumptions, and **add more data** if you think it makes for a more complete solution.

Minimum Requirements

1. Chatbot and Natural-Language Queries

Build at least one chatbot that accepts natural-language requests. It may be customer-facing or designed for authorised ParcelPilot support/operations staff. The chatbot should use only the supplied data pack as its information base. Different sources may have different levels of authority, freshness, and reliability, which the system should account for when answering.

Queries that can be answered confidently from the available sources should be handled directly. Queries that require human judgment, unsupported exceptions, or actions outside the system's capabilities should be escalated to the support team.

2. Access Control and Data Privacy

The underlying data may be used in customer-facing and authorised internal contexts. If you build a customer-facing chatbot, customers must only be able to access data belonging to their own account. If you build an internal chatbot, access should be limited to authorised ParcelPilot users and scoped appropriately for their role.

You may mock authentication, account context, and user roles as appropriate for the chatbot you choose to build.

Access controls should be enforced in the data/tool layer rather than relying only on model instructions. Sensitive customer information must not be exposed to unauthorised users.

3. Agent Tools

The agent must have at least three distinct tools that it can choose between:

1. **Document search/retrieval**
Search policies, agreements, product documentation, SOPs, and other supplied documents.
2. **Structured-data lookup or calculation**
Query or calculate information using the supplied account, order, and ticket data.
3. **State-changing action**
Perform an action such as:
 - Creating an escalation
 - Updating a ticket
 - Creating a follow-up task

The action tool may be mocked locally.

4. Confirmation Before Actions

Any state-changing action must require **explicit user confirmation** before it is executed.

For example, if the user asks to escalate a ticket, the agent may prepare the escalation but should ask for confirmation before actually creating it.

5. Multi-Step Requests

The system must support requests that require multiple tools or sources to answer.

For example, answering a question may require:

- Looking up an order
- Identifying the customer's account
- Reading the customer's agreement
- Checking the applicable policy or SOP
- Performing a calculation
- Deciding whether an escalation or other action is required

6. Interface

Expose your chosen chatbot through a simple chat interface. The interface should ideally show which tool is being used.

Hosting the complete application and submitting a hosted link is highly preferred.

7. Demo Video

Submit a video of approximately **5 minutes** covering:

- The solution architecture
- A demonstration of the working application
- Important product or technical decisions you made, along with reason

Example Requests

Your system should be able to handle questions such as:

- **Can Northstar cancel ORD-1001 without a cancellation fee? Explain why.**
- **A pickup is three hours late because of carrier fault. Should I get a service credit?**

These examples are illustrative.

We may test your system using other records and questions from the same source pack.

Your implementation should therefore load and reason over the supplied data rather than hard-coding the example IDs or answers.

Two Additional Client Problems

Beyond the minimum requirements, ParcelPilot has identified two broader problems.

Problem 1: Proactive Issue Detection

A reactive chatbot only helps once someone asks a question.

ParcelPilot also wants an internal view for authorised support and operations users that identifies recurring, urgent, or unusual issues across its support activity and helps the team understand what deserves attention.

For example, the system might identify:

- A sudden increase in similar customer complaints
- Multiple tickets related to the same product issue
- High-severity tickets approaching or exceeding SLA
- Unusual patterns within orders or support activity
- Issues affecting multiple customers at the same time

Problem 2: Trust and Reliability

The team is concerned about trust.

Policies change, customer contracts may override general rules, different systems may disagree, and previous support answers may be wrong.

A confidently incorrect answer or action would quickly reduce adoption.

The system should therefore make deliberate decisions about source reliability, conflicts, uncertainty, and when human intervention is appropriate.

What We Want From You

Required

Build the minimum requirements described above.

Think Beyond the Immediate Requirements

Tell us what else you would develop for ParcelPilot if you were continuing to work on the product.

Prioritise your ideas and explain why they matter.

We are intentionally not specifying every aspect of the workflow, architecture, or interface. Use your judgment and make sensible product and technical trade-offs.

Submission Requirements

Please submit the following:

Task Submission form-<https://forms.gle/hLGBrDrNRmK7UAbv6>

1. Repository

A link to the code repository (public repo) with clear setup and run instructions.

2. Hosted Application

A hosted version of the application is highly preferred. If available, include the URL.

3. Demo Video

A roughly 5-minute video covering:

- The solution architecture
- A demo of the working application
- Key product and technical decisions

4. Architecture Note

Include a short architecture note covering:

- Agent design
- Tool design

- Document and structured-data handling
- Source reliability and conflict handling
- Major technical trade-offs

5. Product Note

Include a short product note covering:

- Which additional client problem you chose and how you addressed it
- Anything else you would build for ParcelPilot
- What you intentionally left out of the submission
- One metric you would use to judge whether the product is useful

6. AI Tool Usage

Briefly state which AI coding tools you used and how you used them.
